

DBMS FINAL REPORT

Airport Management System

Group Members:

B.GOPI CHAND	142401001
K.VISHNU VARDHAN	142401018
N.SAKETH	142401025
V.SAI YASASWI	142401042

Contents

1	Project Description	3
2	Data Being Dealt With	3
3	Relational Design of the Database	4
3.1	ER Diagram	5
3.2	Relationships Summary	5
3.3	Aggregations	6
4	Table Creation in pgAdmin	6
4.1	Passenger Table	6
4.2	Staff Table	7
4.3	Airports Table	7
4.4	Airroutes Table	8
4.5	Gates Table	9
4.6	Flights Table	10
4.7	Scheduled Flight Table	10
4.8	Bookings Table	11
4.9	Boarding Pass Table	11
4.10	Luggage Table	12
4.11	Staff Manages Table	13
5	Integrity Constraints and General Constraints	13
5.1	Primary Key Constraints	13
5.2	Foreign Key Constraints	14
5.3	UNIQUE Constraints	15
5.4	NOT NULL Constraints	15
5.5	General Constraints (enforced via Triggers)	15

6	Functions and Triggers	15
6.1	List of All Functions	15
6.2	Core Functions — Detailed Implementation	17
6.3	List of All Triggers	21
6.4	Staff Role Management Triggers — SQL Implementation	23
7	Roles and Privileges	26
7.1	Role Descriptions	26
7.2	Table-Level Privilege Matrix	26
7.3	Function Execution Privileges	27
7.4	Role Creation SQL	27
8	Views and Justification	28
8.1	View 1: PassengerBookingSummary	28
8.2	View 2: FlightGateStatus	28
8.3	View 3: LuggageSecurityView	29
8.4	View 4: StaffFlightView	29
8.5	View 5: FlightRouteView	30
8.6	View 6: BookingDetailsView	31
8.7	View 7: LuggageSummaryView	31
9	Indices	32
9.1	Index Types Used	32
9.2	B-Tree Indices	32
9.3	Hash Indices	33
9.4	GIN Indices	33
9.5	Index Summary Table	34
10	Five Queries Using Functions, Roles, and Indices	34
10.1	Query 1: Passenger Check-In Lookup	34
10.2	Query 2: Daily Flight Schedule + Booking Count	35
10.3	Query 3: Security Luggage Inspection	37
10.4	Query 4: Gate Management and Flight Scheduling	39
10.5	Query 5: Staff Assignment and Flight Status Update	41
11	Primary Tasks on the Data	44
	Index	45

1 Project Description

The Airport Management System is a database-driven application designed to manage and automate essential airport operations such as passenger handling, flight scheduling, gate allocation, luggage tracking, and crowd monitoring. The system stores and processes data related to passengers, staff, flights, airports, routes, bookings, boarding passes, and luggage. It supports both online and offline ticket bookings by verifying existing booking records at check-in. Once a booking is confirmed, the system automatically generates a boarding pass by assigning seat number, gate number, and boarding time. The system enforces data integrity and improves operational efficiency through proper use of DBMS concepts such as primary keys, foreign keys, constraints, and triggers.

2 Data Being Dealt With

- Passenger personal and travel details
- Staff login, role, and address information
- Flight schedules, capacity, and gate assignment data
- Airport and air route information (origin, destination, distance)
- Ticket booking records (online and offline)
- Boarding pass and gate allocation data
- Luggage details and security clearance status

3 Relational Design of the Database

1. **Passenger**
 - Passenger_ID (PK)
 - Name (NOT NULL)
 - Email (UNIQUE)
 - Phone
 - Passport_Number (UNIQUE)
2. **Staff**
 - Staff_ID (PK)
 - Name (NOT NULL)
 - Username (UNIQUE)
 - Password_Hash
 - Role (Admin, Security, Ground_Staff)
 - Street, City, State (Composite Address)
3. **Airport**
 - Airport_ID (PK)
 - Airport_Name
 - City, State, Country
4. **Airroutes**
 - Route_ID (PK)
 - Origin_Airport (FK → Airport)
 - Destination_Airport (FK → Airport)
 - Distance_KM
5. **Gates**
 - Gate_ID (PK)
 - Gate_Number
 - Airport_ID (FK → Airport)
 - Status (Available, Occupied, Maintenance)
6. **Flights**
 - Flight_ID (PK)
 - Flight_No (UNIQUE)
 - Aircraft_Type
 - Capacity
7. **Scheduled_Flight (Aggregation)**
 - Route_ID (FK → Airroutes)
 - Flight_ID (FK → Flights)
 - Flight_Date, Departure_Time, Arrival_Time
 - Gate_ID (FK → Gates)
 - Status
 - PK(Route_ID, Flight_ID, Flight_Date)
8. **Bookings**
 - Booking_ID (PK)
 - Passenger_ID (FK → Passenger)
 - Route_ID, Flight_ID, Flight_Date
 - Booking_Date, Booking_Status
 - Traveller_Name, Traveller_Email
 - Traveller_Phone, Traveller_Passport
 - Group_ID (for group bookings)
9. **Boarding_Pass (Weak Entity)**
 - Booking_ID (PK, FK → Bookings)
 - Seat_Number, Boarding_Group, Boarding_Time
10. **Luggage**
 - Luggage_ID (PK)
 - Booking_ID (FK → Bookings)
 - Weight, Security_Status, Checkin_Time
11. **Staff_Manages**
 - Staff_ID (FK → Staff)
 - Route_ID, Flight_ID, Flight_Date
 - PK(Staff_ID, Route_ID, Flight_ID, Flight_Date)

3.1 ER Diagram

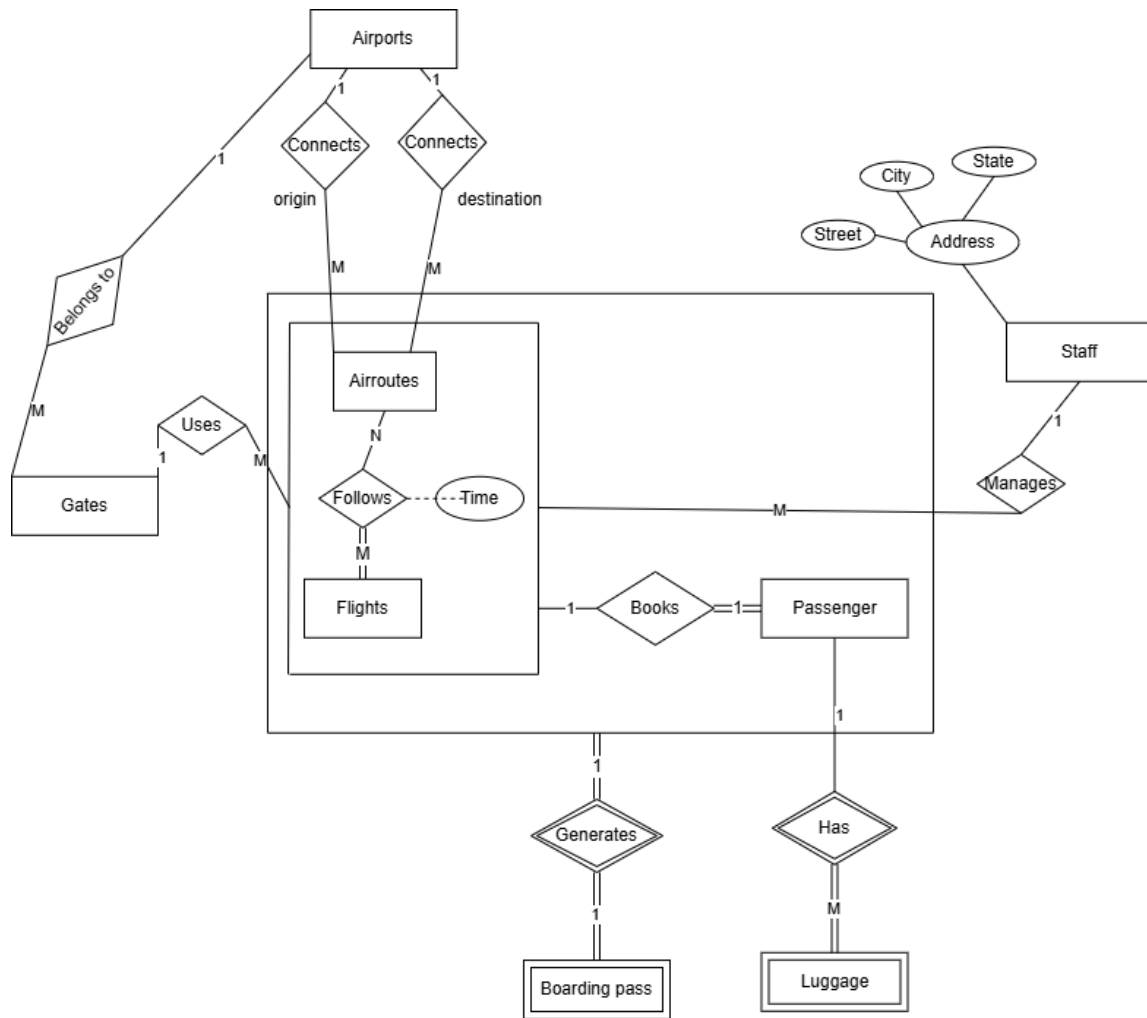


Figure 1: Entity Relationship Diagram for Airport Management System

3.2 Relationships Summary

- Airports 1–M Airroutes via Origin FK (Connects — origin side)
- Airports 1–M Airroutes via Destination FK (Connects — destination side)
- Airports 1–M Gates (via Airport_ID FK in Gates) — Belongs To
- Airroutes M–N Flights (resolved via Scheduled_Flight with Time attribute) — Follows
- Aggregation 1 (Airroutes + Follows + Flights) M–1 Gates (via Gate_ID FK) — Uses
- Passenger 1–M Bookings via Books relationship
- Bookings 1–1 Boarding_Pass — Generates
- Passenger 1–M Luggage (via Booking_ID FK) — Has
- Staff 1–M Scheduled_Flight via Staff_Manages — Manages

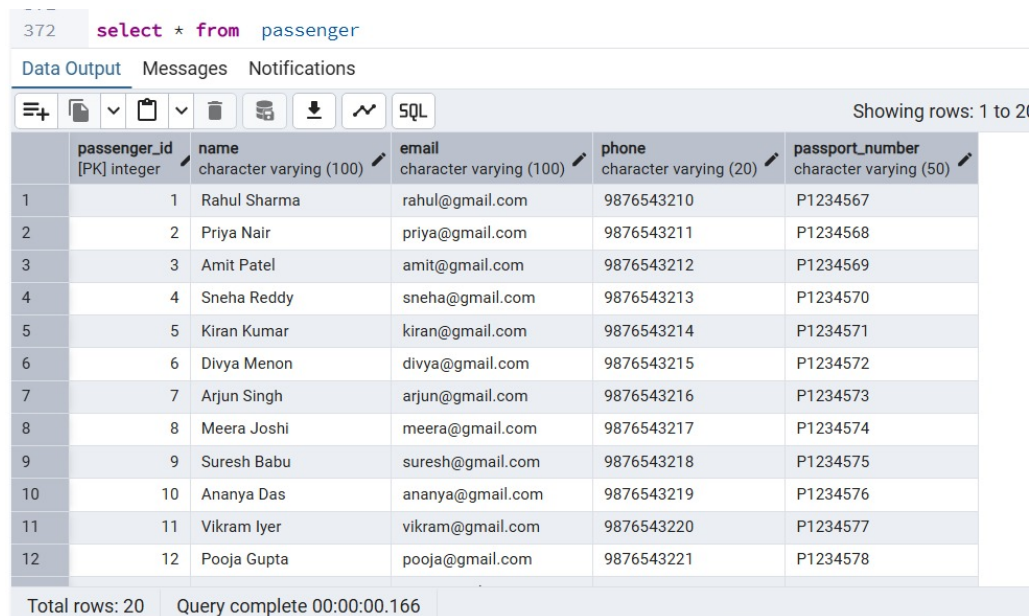
3.3 Aggregations

- **Aggregation 1 — Inner box (Airroutes + Follows + Flights):** The N:M relationship *Follows* between Airroutes and Flights is treated as a single aggregated unit. This captures the fact that a flight operates on a specific air route at a scheduled time. This aggregated unit is then used as a participant in the outer aggregation.
- **Aggregation 2 — Outer box (Aggregation 1 + Books + Passenger):** The inner aggregated unit (Airroutes + Follows + Flights) together with the Passenger entity, connected through the *Books* relationship, forms the outer aggregation. This entire outer aggregation then participates in the *Generates* relationship, producing a **Boarding Pass**. This design correctly models the fact that a boarding pass is generated only when a specific passenger books a specific flight on a specific route — not from any single entity alone.

4 Table Creation in pgAdmin

All relational tables were created using **PostgreSQL** via the **pgAdmin** interface. Each table was defined with appropriate primary keys, foreign key constraints, unique constraints, and NOT NULL conditions. The following screenshots show the successful creation of each table in the database.

4.1 Passenger Table

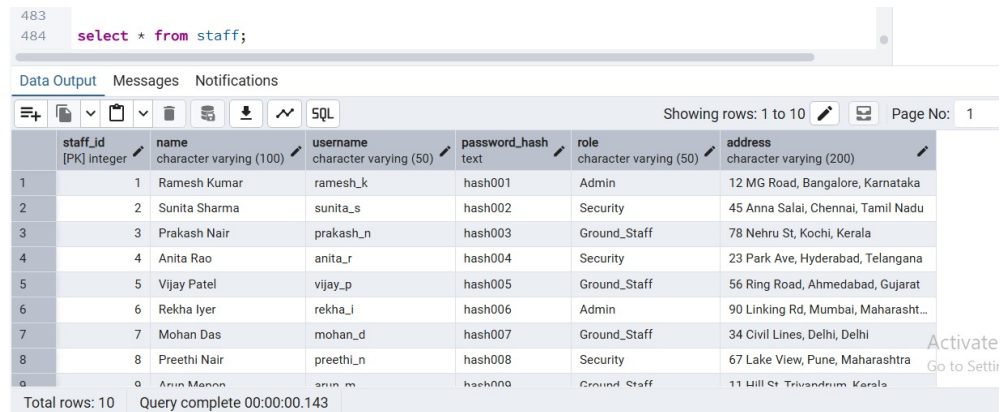


	passenger_id [PK] integer	name character varying (100)	email character varying (100)	phone character varying (20)	passport_number character varying (50)
1	1	Rahul Sharma	rahul@gmail.com	9876543210	P1234567
2	2	Priya Nair	priya@gmail.com	9876543211	P1234568
3	3	Amit Patel	amit@gmail.com	9876543212	P1234569
4	4	Sneha Reddy	sneha@gmail.com	9876543213	P1234570
5	5	Kiran Kumar	kiran@gmail.com	9876543214	P1234571
6	6	Divya Menon	divya@gmail.com	9876543215	P1234572
7	7	Arjun Singh	arjun@gmail.com	9876543216	P1234573
8	8	Meera Joshi	meera@gmail.com	9876543217	P1234574
9	9	Suresh Babu	suresh@gmail.com	9876543218	P1234575
10	10	Ananya Das	ananya@gmail.com	9876543219	P1234576
11	11	Vikram Iyer	vikram@gmail.com	9876543220	P1234577
12	12	Pooja Gupta	pooja@gmail.com	9876543221	P1234578

Total rows: 20 Query complete 00:00:00.166

Figure 2: Creation of the Passenger table in pgAdmin

4.2 Staff Table



483
484 `select * from staff;`

Data Output Messages Notifications

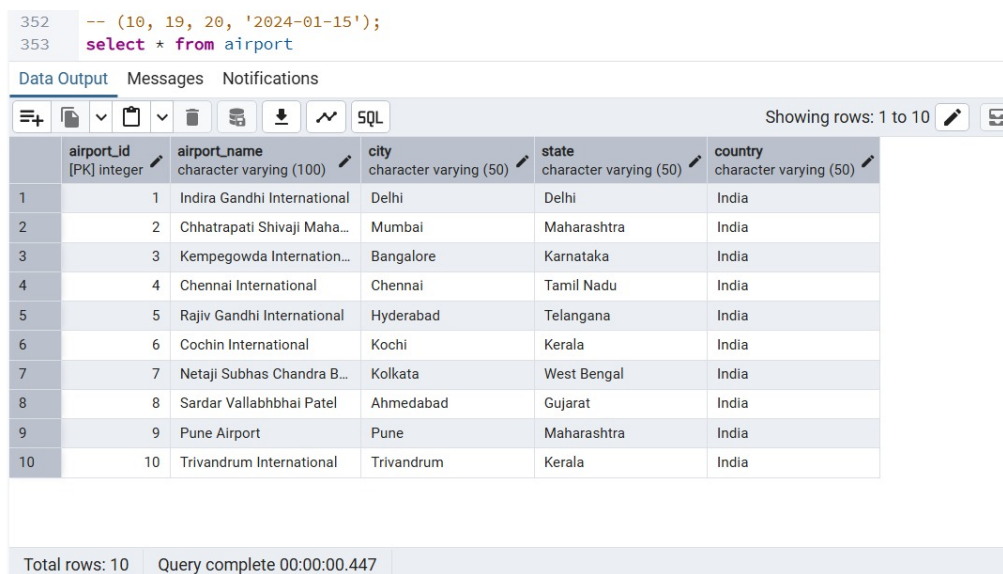
Showing rows: 1 to 10 Page No: 1

	staff_id [PK] integer	name character varying (100)	username character varying (50)	password_hash text	role character varying (50)	address character varying (200)
1	1	Ramesh Kumar	ramesh_k	hash001	Admin	12 MG Road, Bangalore, Karnataka
2	2	Sunita Sharma	sunita_s	hash002	Security	45 Anna Salai, Chennai, Tamil Nadu
3	3	Prakash Nair	prakash_n	hash003	Ground_Staff	78 Nehru St, Kochi, Kerala
4	4	Anita Rao	anita_r	hash004	Security	23 Park Ave, Hyderabad, Telangana
5	5	Vijay Patel	vijay_p	hash005	Ground_Staff	56 Ring Road, Ahmedabad, Gujarat
6	6	Rekha Iyer	rekha_i	hash006	Admin	90 Linking Rd, Mumbai, Maharasht...
7	7	Mohan Das	mohan_d	hash007	Ground_Staff	34 Civil Lines, Delhi, Delhi
8	8	Preethi Nair	preethi_n	hash008	Security	67 Lake View, Pune, Maharashtra
9	9	Anus Menon	anus_m	hash009	Ground_Staff	11 Hill St Trivandrum, Kerala

Total rows: 10 Query complete 00:00:00.143

Figure 3: Creation of the Staff table in pgAdmin

4.3 Airports Table



352 `-- (10, 19, 20, '2024-01-15');`
353 `select * from airport`

Data Output Messages Notifications

Showing rows: 1 to 10

	airport_id [PK] integer	airport_name character varying (100)	city character varying (50)	state character varying (50)	country character varying (50)
1	1	Indira Gandhi International	Delhi	Delhi	India
2	2	Chhatrapati Shivaji Maha...	Mumbai	Maharashtra	India
3	3	Kempegowda Internation...	Bangalore	Karnataka	India
4	4	Chennai International	Chennai	Tamil Nadu	India
5	5	Rajiv Gandhi International	Hyderabad	Telangana	India
6	6	Cochin International	Kochi	Kerala	India
7	7	Netaji Subhas Chandra B...	Kolkata	West Bengal	India
8	8	Sardar Vallabhbhai Patel	Ahmedabad	Gujarat	India
9	9	Pune Airport	Pune	Maharashtra	India
10	10	Trivandrum International	Trivandrum	Kerala	India

Total rows: 10 Query complete 00:00:00.447

Figure 4: Creation of the Airports table in pgAdmin

4.4 Airroutes Table

355
356
357 `select * from airroutes`

Data Output Messages Notifications

	route_id [PK] integer	origin_airport integer	destination_airport integer	distance_km integer
1	1	1	2	1150
2	2	1	3	1740
3	3	1	4	2180
4	4	1	5	1250
5	5	2	3	980
6	6	2	4	1330
7	7	2	6	1200
8	8	3	4	1060
9	9	3	5	570
10	10	4	5	630
11	11	4	6	690
12	12	5	6	740

Total rows: 20 Query complete 00:00:00.211

Figure 5: Creation of the `Airroutes` table in pgAdmin

4.5 Gates Table

```
365 select * from gates |
366
```

Data Output Messages Notifications

	gate_id [PK] integer	gate_number character varying (10)	status character varying (20)
1	1	A1	Available
2	2	A2	Occupied
3	3	A3	Maintenance
4	4	B1	Available
5	5	B2	Occupied
6	6	B3	Available
7	7	C1	Available
8	8	C2	Occupied
9	9	D1	Available
10	10	D2	Maintenance
11	11	E1	Available
12	12	E2	Occupied

Figure 6: Creation of the `Gates` table in pgAdmin

4.6 Flights Table

362
363 `select * from flights`

Data Output Messages Notifications

SQL

	flight_id [PK] integer	flight_no character varying (20)	aircraft_type character varying (50)	capacity integer
1	1	AI101	Boeing 737	180
2	2	AI102	Airbus A320	160
3	3	AI103	Boeing 777	300
4	4	AI104	Airbus A380	400
5	5	AI105	Boeing 737	180
6	6	AI106	Airbus A320	160
7	7	AI107	Boeing 737	180
8	8	AI108	Airbus A321	200
9	9	AI109	Boeing 737	180
10	10	AI110	Airbus A320	160
11	11	6E201	Airbus A320	180
12	12	6E202	Airbus A321	200

Total rows: 20 Query complete 00:00:00.111

Figure 7: Creation of the `Flights` table in pgAdmin

4.7 Scheduled Flight Table

374 `select * from scheduled_flight`

Data Output Messages Notifications

SQL

Showing rows: 1 to 21

	route_id [PK] integer	flight_id [PK] integer	flight_date [PK] date	departure_time time without time zone	arrival_time time without time zone	gate_id integer	status character varying (20)
1	1	1	2024-01-15	06:00:00	08:00:00	1	On_Time
2	1	2	2024-01-15	09:00:00	11:30:00	2	On_Time
3	2	3	2024-01-15	07:00:00	09:45:00	7	Delayed
4	3	4	2024-01-15	10:00:00	14:00:00	9	On_Time
5	4	5	2024-01-15	11:00:00	13:30:00	11	On_Time
6	5	6	2024-01-15	08:00:00	09:45:00	4	On_Time
7	6	7	2024-01-15	13:00:00	15:20:00	13	Cancelled
8	7	8	2024-01-15	14:00:00	17:00:00	15	On_Time
9	8	9	2024-01-15	06:30:00	08:15:00	7	On_Time
10	9	10	2024-01-15	09:30:00	11:00:00	9	Delayed
11	10	11	2024-01-15	16:00:00	17:10:00	11	On_Time
12	11	12	2024-01-15	17:00:00	19:30:00	13	On_Time

Total rows: 21 Query complete 00:00:00.196

Figure 8: Creation of the `Scheduled_Flight` table in pgAdmin

4.8 Bookings Table

361 `select * from bookings`

Data Output Messages Notifications

Showing rows: 1 to 20

	booking_id [PK] integer	passenger_id integer	route_id integer	flight_id integer	flight_date date	booking_date date	booking_status character varying (30)
1	1	1	1	1	2024-01-15	2024-01-01	Confirmed
2	2	2	1	2	2024-01-15	2024-01-02	Confirmed
3	3	3	2	3	2024-01-15	2024-01-02	Confirmed
4	4	4	3	4	2024-01-15	2024-01-03	Cancelled
5	5	5	4	5	2024-01-15	2024-01-03	Confirmed
6	6	6	5	6	2024-01-15	2024-01-04	Confirmed
7	7	7	6	7	2024-01-15	2024-01-04	Confirmed
8	8	8	7	8	2024-01-15	2024-01-05	Confirmed
9	9	9	8	9	2024-01-15	2024-01-05	Confirmed
10	10	10	9	10	2024-01-15	2024-01-06	Confirmed
11	11	11	10	11	2024-01-15	2024-01-06	Confirmed
12	12	12	11	12	2024-01-15	2024-01-07	Confirmed

Total rows: 20 Query complete 00:00:00.145

Figure 9: Creation of the Bookings table in pgAdmin

4.9 Boarding Pass Table

358
359 `select * from boarding_pass`

Data Output Messages Notifications

Showing rows: 1 to 18

	booking_id [PK] integer	seat_number character varying (10)	boarding_group character varying (10)	boarding_time time without time zone
1	1	12A	Group1	05:30:00
2	2	14B	Group2	08:30:00
3	3	22C	Group1	06:30:00
4	5	8D	Group1	10:30:00
5	6	5A	Group2	07:30:00
6	7	18B	Group3	12:30:00
7	8	25C	Group1	13:30:00
8	9	3A	Group2	06:00:00
9	10	11D	Group3	09:00:00
10	11	7B	Group1	15:30:00
11	12	19A	Group2	16:30:00
12	13	2C	Group1	17:30:00

Total rows: 18 Query complete 00:00:00.127

Figure 10: Creation of the Boarding_Pass table in pgAdmin

4.10 Luggage Table

370 `select * from luggage`

Data Output Messages Notifications

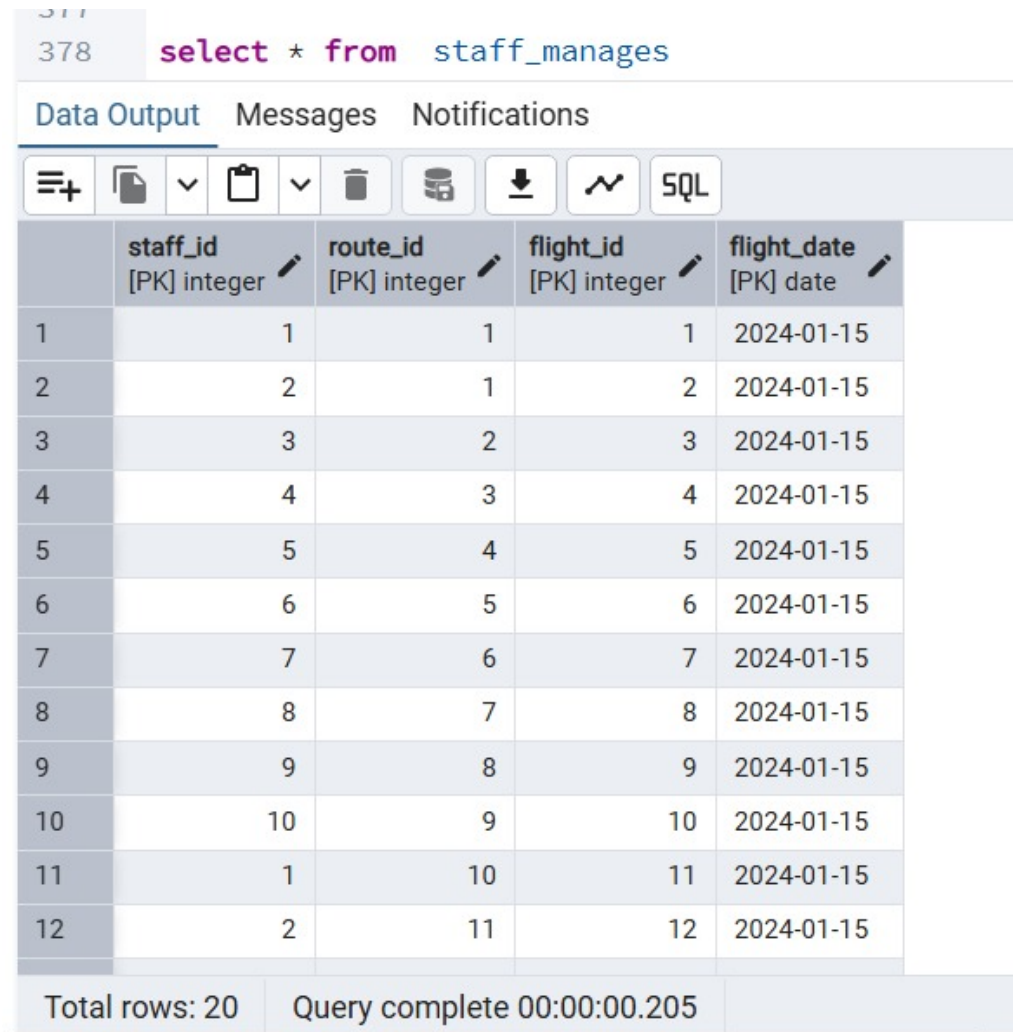
SQL Show

	<code>luggage_id</code> [PK] integer	<code>booking_id</code> integer	<code>weight</code> numeric (5,2)	<code>security_status</code> character varying (20)	<code>checkin_time</code> timestamp without time zone
1	1	1	23.50	Cleared	2024-01-15 04:30:00
2	2	2	18.00	Cleared	2024-01-15 07:45:00
3	3	3	30.00	Flagged	2024-01-15 05:30:00
4	4	5	15.50	Cleared	2024-01-15 09:00:00
5	5	6	22.00	Cleared	2024-01-15 06:30:00
6	6	7	10.00	Cleared	2024-01-15 11:00:00
7	7	8	28.50	Cleared	2024-01-15 12:00:00
8	8	9	20.00	Cleared	2024-01-15 05:00:00
9	9	10	17.50	Flagged	2024-01-15 08:00:00
10	10	11	25.00	Cleared	2024-01-15 14:30:00
11	11	12	19.00	Cleared	2024-01-15 15:30:00
12	12	13	12.50	Cleared	2024-01-15 16:30:00

Total rows: 18 Query complete 00:00:00.180

Figure 11: Creation of the Luggage table in pgAdmin

4.11 Staff Manages Table



	staff_id [PK] integer	route_id [PK] integer	flight_id [PK] integer	flight_date [PK] date
1	1	1	1	2024-01-15
2	2	1	2	2024-01-15
3	3	2	3	2024-01-15
4	4	3	4	2024-01-15
5	5	4	5	2024-01-15
6	6	5	6	2024-01-15
7	7	6	7	2024-01-15
8	8	7	8	2024-01-15
9	9	8	9	2024-01-15
10	10	9	10	2024-01-15
11	1	10	11	2024-01-15
12	2	11	12	2024-01-15

Total rows: 20 Query complete 00:00:00.205

Figure 12: Creation of the Staff_Manages table in pgAdmin

5 Integrity Constraints and General Constraints

5.1 Primary Key Constraints

Every table has a primary key that uniquely identifies each row. Serial primary keys are auto-generated by PostgreSQL.

Table	Primary Key
Passenger	Passenger_ID (SERIAL)
Staff	Staff_ID (SERIAL)
Airport	Airport_ID (SERIAL)
Airroutes	Route_ID (SERIAL)
Gates	Gate_ID (SERIAL)
Flights	Flight_ID (SERIAL)
Scheduled_Flight	(Route_ID, Flight_ID, Flight_Date) composite
Bookings	Booking_ID (SERIAL)
Boarding_Pass	Booking_ID (references Bookings)
Luggage	Luggage_ID (SERIAL)
Staff_Manages	(Staff_ID, Route_ID, Flight_ID, Flight_Date) composite

5.2 Foreign Key Constraints

Column	References	Business Rule
Airroutes.Origin_Airport	Airport(Airport_ID)	Route must start from a valid airport
Airroutes.Destination_Airport	Airport(Airport_ID)	Route must end at a valid airport
Gates.Airport_ID	Airport(Airport_ID)	Gate must belong to a valid airport
Scheduled_Flight.Route_ID	Airroutes(Route_ID)	Flight must follow a valid route
Scheduled_Flight.Flight_ID	Flights(Flight_ID)	Scheduled flight must be a valid flight
Scheduled_Flight.Gate_ID	Gates(Gate_ID)	Flight must be assigned to a valid gate
Bookings.Passenger_ID	Passenger(Passenger_ID)	Booking must belong to a valid passenger
Bookings.(Route,Flight,Date)	Scheduled_Flight	Booking must reference a scheduled flight
Bookings.Group_ID	(self-referencing INT)	Groups multiple travellers under one booking group
Boarding_Pass.Booking_ID	Bookings(Booking_ID)	Boarding pass must reference a valid booking
Luggage.Booking_ID	Bookings(Booking_ID)	Luggage must be linked to a valid booking
Staff_Manages.Staff_ID	Staff(Staff_ID)	Only valid staff can manage flights
Staff_Manages.(Route,Flight,Date)	Scheduled_Flight	Must manage an existing scheduled flight

5.3 UNIQUE Constraints

Column	Justification
Passenger.Email	No two passengers can share the same email address
Passenger.Passport_Number	Passport numbers are globally unique identifiers
Staff.Username	Each staff member must have a distinct login username
Flights.Flight_No	Flight numbers are unique airline identifiers

5.4 NOT NULL Constraints

Business-critical columns carry explicit NOT NULL constraints to ensure essential data is never missing:

- **Passenger:** Name
- **Staff:** Name
- **Scheduled_Flight:** Flight_Date, Departure_Time, Arrival_Time, Status
- **Bookings:** Booking_Date, Booking_Status, Traveller_Name
- **Luggage:** Weight, Security_Status

5.5 General Constraints (enforced via Triggers)

- Luggage weight must not exceed 30 kg (`trg_luggage_weight`)
- A passenger cannot book the same flight twice on the same date (`trg_prevent_double_booking`)
- A gate under maintenance cannot be assigned to any flight (`trg_gate_maintenance`)
- Bookings cannot be made on cancelled or departed flights (`trg_flight_departure_check`)
- Only Ground_Staff or Admin can be assigned to manage flights (`trg_staff_role_check`)
- Duplicate passport numbers are blocked at insert time (`trg_passport_duplicate`)
- Gate conflicts at overlapping times are prevented (`trg_gate_conflict`)
- Flight capacity is validated before each booking (`trg_check_capacity`)

6 Functions and Triggers

The system implements **32 functions/procedures** and **14 triggers** written in PL/pgSQL. Together they enforce atomicity, validate business rules, maintain audit trails, and preserve database consistency.

6.1 List of All Functions

#	Function Name	Type	Purpose	Accessible By
1	RegisterPassenger	Procedure	Register new passenger with duplicate checks	admin, checkin_staff

#	Function Name	Type	Purpose	Accessible By
2	GetPassengerByID	Function	Fetch passenger record by ID	admin, checkin_staff
3	UpdatePassenger	Procedure	Update passenger details using COALESCE	admin, checkin_staff
4	SearchPassenger	Function	ILIKE search by name, email or passport	admin, checkin_staff
5	StaffLogin	Function	Authenticate staff by username and password hash	All roles
6	SearchStaff	Function	ILIKE search staff by name, role or username	admin
7	GetAllAirports	Function	List all airports alphabetically	admin, ground_staff
8	SearchAirport	Function	ILIKE search airports by name, city or country	admin, ground_staff, checkin_staff
9	GetRoutesByOrigin	Function	Get all routes from a given origin airport	admin, ground_staff
10	SearchRoutes	Function	Find routes by origin and destination pair	admin, ground_staff
11	GetGatesByAirport	Function	List gates for a given airport	admin, ground_staff
12	UpdateGateStatus	Procedure	Set gate status to Available/Occupied/Maintenance	admin, ground_staff
13	SearchGate	Function	Filter gates by airport and/or status	admin, ground_staff
14	GetAllFlights	Function	List all flights ordered by flight number	admin, ground_staff
15	SearchFlight	Function	Search flights by flight number or aircraft type	admin, ground_staff
16	ScheduleFlight	Procedure	Schedule a flight on a route with gate check	admin
17	GetFlightsByDate	Function	Get all scheduled flights on a specific date	admin, ground_staff
18	UpdateFlightStatus	Procedure	Update scheduled flight status with validation	admin, ground_staff
19	SearchScheduledFlight	Function	Flexible search by date, status or route	admin, ground_staff
20	BookFlight	Function	Create a confirmed booking for a passenger	checkin_staff, admin
21	CancelBooking	Procedure	Cancel a booking and trigger boarding pass removal	checkin_staff, admin

#	Function Name	Type	Purpose	Accessible By
22	GetBookingsByPassenger	Function	Get all bookings for a passenger with route info	checkin_staff, admin
23	SearchBooking	Function	Search bookings by passenger, status or date	checkin_staff, admin
24	GetBoardingPass	Function	Fetch full boarding pass details by booking ID	checkin_staff, security_staff, admin
25	AddLuggage	Procedure	Check in luggage with weight validation	checkin_staff, admin
26	UpdateLuggageStatus	Function	Set luggage security status to Cleared or Flagged	security_staff, admin
27	GetLuggageByBooking	Function	Get all luggage records for a booking	security_staff, checkin_staff, admin
28	SearchLuggage	Function	Search luggage by booking ID and/or status	security_staff, admin
29	AssignGate	Function	Assign gate to a flight and mark gate as Occupied	admin, ground_staff
30	AssignStaffToFlight	Procedure	Assign a staff member to manage a scheduled flight	admin
31	SearchStaffFlight	Function	Search staff-flight assignments by staff or date	admin, ground_staff
32	AssignStaffToFlight (role check)	(role —	Works with trg_staff_role_check for two-layer enforcement	admin

6.2 Core Functions — Detailed Implementation

The following five functions represent the core passenger-facing and operational workflows of the system. Full SQL implementations are provided below.

Core Function 1: RegisterPassenger (Procedure)

Purpose: Registers a new passenger. Validates that the email and passport number are not already in use. Raises a descriptive exception for duplicates.

Consistency Role: Prevents duplicate passenger records, preserving the integrity of the Passenger table.

Accessible By: admin, checkin_staff

```
CREATE OR REPLACE PROCEDURE RegisterPassenger(
    p_name    VARCHAR(100),
    p_email   VARCHAR(100),
```

```

    p_phone    VARCHAR(15),
    p_passport VARCHAR(20)
)
LANGUAGE plpgsql AS $$
DECLARE
    new_id INT;
BEGIN
    IF EXISTS (SELECT 1 FROM Passenger
               WHERE Email = p_email) THEN
        RAISE EXCEPTION 'Error: Email already registered.';
    ELSIF EXISTS (SELECT 1 FROM Passenger
                 WHERE Passport_Number = p_passport) THEN
        RAISE EXCEPTION
            'Error: Passport number already registered.';
    ELSE
        INSERT INTO Passenger(Name, Email, Phone, Passport_Number)
        VALUES (p_name, p_email, p_phone, p_passport)
        RETURNING Passenger_ID INTO new_id;
        RAISE NOTICE 'New Passenger ID: %', new_id;
    END IF;
END;
$$;

```

Listing 1: RegisterPassenger procedure

Core Function 2: BookFlight (Function)

Purpose: Creates a confirmed booking for a passenger on a scheduled flight. Supports traveller details and group bookings via `group_id`. Returns the new Booking_ID.

Consistency Role: Works with `trg_check_capacity` and `trg_prevent_double_booking` triggers to enforce flight-level constraints. Also fires `trg_generate_boarding_pass` automatically after insert.

Accessible By: `checkin_staff`, `admin`

```

CREATE OR REPLACE FUNCTION BookFlight(
    p_passenger_id INT,
    p_route_id     INT,
    p_flight_id    INT,
    p_flight_date  DATE,
    p_traveller_name VARCHAR(100),
    p_traveller_email VARCHAR(100),
    p_traveller_phone VARCHAR(20),
    p_traveller_passport VARCHAR(20),
    p_group_id     INT DEFAULT NULL
)
RETURNS INT AS $$
DECLARE
    new_booking_id INT;
BEGIN
    INSERT INTO Bookings(

```

```

    Passenger_ID, Route_ID, Flight_ID, Flight_Date,
    Booking_Date, Booking_Status,
    Traveller_Name, Traveller_Email,
    Traveller_Phone, Traveller_Passport,
    Group_ID
)
VALUES (
    p_passenger_id, p_route_id, p_flight_id, p_flight_date,
    CURRENT_DATE, 'Confirmed',
    p_traveller_name, p_traveller_email,
    p_traveller_phone, p_traveller_passport,
    p_group_id
)
RETURNING Booking_ID INTO new_booking_id;
RETURN new_booking_id;
END;
$$ LANGUAGE plpgsql;

```

Listing 2: BookFlight function

Core Function 3: GetBoardingPass (Function)

Purpose: Retrieves the complete boarding pass for a passenger given their Booking_ID. Joins across 8 tables to return a single comprehensive record.

Consistency Role: Provides all boarding information in one call, reducing risk of inconsistent partial reads across tables.

Accessible By: checkin_staff, security_staff, admin

```

CREATE OR REPLACE FUNCTION GetBoardingPass(p_booking_id INT)
RETURNS TABLE(
    Booking_ID INT, Passenger_Name VARCHAR,
    Passport_Number VARCHAR, Flight_No VARCHAR,
    From_Airport VARCHAR, To_Airport VARCHAR,
    Flight_Date DATE, Departure_Time TIME,
    Arrival_Time TIME, Gate_Number VARCHAR,
    Seat_Number VARCHAR, Boarding_Group VARCHAR,
    Boarding_Time TIME
) AS $$
BEGIN
    RETURN QUERY
    SELECT bp.Booking_ID, p.Name, p.Passport_Number,
           f.Flight_No, a1.Airport_Name, a2.Airport_Name,
           sf.Flight_Date, sf.Departure_Time, sf.Arrival_Time,
           g.Gate_Number, bp.Seat_Number,
           bp.Boarding_Group, bp.Boarding_Time
    FROM Boarding_Pass bp
    JOIN Bookings b      ON bp.Booking_ID = b.Booking_ID
    JOIN Passenger p     ON b.Passenger_ID = p.Passenger_ID
    JOIN Flights f       ON b.Flight_ID   = f.Flight_ID
    JOIN Scheduled_Flight sf ON b.Flight_ID = sf.Flight_ID

```

```

                                AND b.Route_ID = sf.Route_ID
                                AND b.Flight_Date = sf.Flight_Date
JOIN Gates g                    ON sf.Gate_ID = g.Gate_ID
JOIN Airroutes ar               ON b.Route_ID = ar.Route_ID
JOIN Airport a1                 ON ar.Origin_Airport
                                = a1.Airport_ID
JOIN Airport a2                 ON ar.Destination_Airport
                                = a2.Airport_ID
WHERE bp.Booking_ID = p_booking_id;
END;
$$ LANGUAGE plpgsql;

```

Listing 3: GetBoardingPass function

Core Function 4: AssignGate (Function)

Purpose: Assigns a gate to a scheduled flight and marks the gate as Occupied.

Consistency Role: Atomically updates both Scheduled_Flight and Gates tables, keeping gate status consistent across both tables in one operation.

Accessible By: admin, ground_staff

```

CREATE OR REPLACE FUNCTION AssignGate(
    p_route_id INT,
    p_flight_id INT,
    p_flight_date DATE,
    p_gate_id INT
)
RETURNS VOID AS $$
BEGIN
    UPDATE Scheduled_Flight SET Gate_ID = p_gate_id
    WHERE Route_ID = p_route_id
        AND Flight_ID = p_flight_id
        AND Flight_Date = p_flight_date;
    UPDATE Gates SET Status = 'Occupied'
    WHERE Gate_ID = p_gate_id;
END;
$$ LANGUAGE plpgsql;

```

Listing 4: AssignGate function

Core Function 5: UpdateLuggageStatus (Function)

Purpose: Updates the security status of a luggage item to Cleared or Flagged after physical inspection by security staff.

Consistency Role: Maintains accurate and current security clearance records for each piece of luggage. Central function for the security_staff role workflow.

Accessible By: security_staff, admin

```

CREATE OR REPLACE FUNCTION UpdateLuggageStatus(

```

```

    p_luggage_id INT,
    p_status     VARCHAR
)
RETURNS VOID AS $$
BEGIN
    UPDATE Luggage SET Security_Status = p_status
    WHERE Luggage_ID = p_luggage_id;
END;
$$ LANGUAGE plpgsql;

```

Listing 5: UpdateLuggageStatus function

6.3 List of All Triggers

#	Trigger Name	Table	When	Trigger Function & Consistency Role
1	trg_generate_boarding_pass	Bookings	AFTER INSERT	<code>generate_boarding_pass()</code> — Auto-generates a boarding pass on every new confirmed booking.
2	trg_check_capacity	Bookings	BEFORE INSERT	<code>check_capacity()</code> — Prevents overbooking by checking booking count against aircraft capacity.
3	trg_free_gate	Scheduled_Flight	AFTER UPDATE	<code>free_gate()</code> — Sets gate to Available when flight status changes to Departed.
4	trg_check_luggage_weight	Luggage	BEFORE INSERT	<code>check_luggage_weight()</code> — Rejects any luggage insert exceeding 30 kg.
5	trg_cancel_boarding_pass	Bookings	AFTER UPDATE	<code>cancel_boarding_pass()</code> — Deletes boarding pass when booking is cancelled, preventing orphaned records.
6	trg_prevent_double_booking	Bookings	BEFORE INSERT	<code>prevent_double_booking()</code> — Blocks same passenger booking same flight on same date twice.
7	trg_log_flight_status	Scheduled_Flight	AFTER UPDATE	<code>log_flight_status_change()</code> — Logs every status change to Flight_Status_Log for audit trail.

#	Trigger Name	Table	When	Trigger Function & Consistency Role
8	trg_check_gate_conflict	Scheduled_Flight	BEFORE INSERT	<code>check_gate_conflict()</code> — Prevents two flights using the same gate in overlapping time slots.
9	trg_set_luggage_checkin	Luggage	BEFORE INSERT	<code>set_luggage_checkin_time()</code> — Auto-sets Checkin_Time to CURRENT_TIMESTAMP.
10	trg_set_booking_date	Bookings	BEFORE INSERT	<code>set_booking_date()</code> — Auto-sets Booking_Date to CURRENT_DATE on every insert.
11	trg_check_staff_role	Staff_Manages	BEFORE INSERT	<code>check_staff_role()</code> — Blocks assignment of Security staff; only Ground_Staff or Admin allowed.
12	trg_check_passport_duplicate	Passenger	BEFORE INSERT	<code>check_passport_duplicate()</code> — Raises descriptive exception on duplicate passport numbers.
13	trg_check_gate_maintenance	Scheduled_Flight	BEFORE INSERT	<code>check_gate_maintenance()</code> — Blocks scheduling a flight on a gate under maintenance.
14	trg_check_flight_booking	Bookings	BEFORE INSERT	<code>check_flight_available_for_booking</code> — Prevents bookings on Cancelled or Departed flights.
15	trg_auto_create_staff_role	Staff	AFTER INSERT	<code>auto_create_staff_role()</code> — Auto-creates a PostgreSQL login role and grants the matching group role when a staff member is inserted.
16	trg_auto_drop_staff_role	Staff	AFTER DELETE	<code>auto_drop_staff_role()</code> — Revokes group role, terminates active sessions, and drops the login role when a staff member is deleted.

#	Trigger Name	Table	When	Trigger Function & Consistency Role
17	trg_auto_update_staff_role	Staff	AFTER UPDATE	auto_update_staff_role() — Revokes old group role and grants new group role when a staff member's role or username is changed.

6.4 Staff Role Management Triggers — SQL Implementation

These three triggers automate PostgreSQL role lifecycle management. Whenever a staff record is inserted, deleted, or updated, the corresponding database login role is created, dropped, or re-granted automatically — eliminating manual role management by the DBA.

```
CREATE OR REPLACE FUNCTION auto_create_staff_role()
RETURNS TRIGGER AS $$
DECLARE
    v_group_role VARCHAR(50);
BEGIN
    v_group_role := CASE NEW.Role
        WHEN 'Admin'      THEN 'admin'
        WHEN 'Ground_Staff' THEN 'ground_staff'
        WHEN 'Security'   THEN 'security_staff'
        WHEN 'Checkin'    THEN 'checkin_staff'
        ELSE NULL
    END;
    IF v_group_role IS NULL THEN
        RAISE EXCEPTION 'Unknown role: %. Use Admin, '
            'Ground_Staff, Security, or Checkin.', NEW.Role;
    END IF;
    IF EXISTS (SELECT 1 FROM pg_roles
        WHERE rolname = NEW.Username) THEN
        RAISE NOTICE 'Login role % already exists.',
            NEW.Username;
    ELSE
        EXECUTE format('CREATE ROLE %I LOGIN PASSWORD %L',
            NEW.Username, NEW.Password_Hash);
        RAISE NOTICE 'Created login role: %', NEW.Username;
    END IF;
    EXECUTE format('GRANT %I TO %I',
        v_group_role, NEW.Username);
    RAISE NOTICE 'Granted role % to %',
        v_group_role, NEW.Username;
    RETURN NEW;
```

```

END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_auto_create_staff_role
AFTER INSERT ON Staff
FOR EACH ROW EXECUTE FUNCTION auto_create_staff_role();

```

Listing 6: trg_auto_create_staff_role — auto-create login role on staff insert

```

CREATE OR REPLACE FUNCTION auto_drop_staff_role()
RETURNS TRIGGER AS $$
DECLARE
    v_group_role VARCHAR(50);
BEGIN
    v_group_role := CASE OLD.Role
        WHEN 'Admin'      THEN 'admin'
        WHEN 'Ground_Staff' THEN 'ground_staff'
        WHEN 'Security'    THEN 'security_staff'
        WHEN 'Checkin'     THEN 'checkin_staff'
        ELSE NULL
    END;
    IF NOT EXISTS (SELECT 1 FROM pg_roles
                   WHERE rolname = OLD.Username) THEN
        RAISE NOTICE 'Login role % does not exist.',
            OLD.Username;
        RETURN OLD;
    END IF;
    IF v_group_role IS NOT NULL THEN
        EXECUTE format('REVOKE %I FROM %I',
            v_group_role, OLD.Username);
        RAISE NOTICE 'Revoked role % from %',
            v_group_role, OLD.Username;
    END IF;
    PERFORM pg_terminate_backend(pid)
    FROM pg_stat_activity WHERE username = OLD.Username;
    EXECUTE format('DROP ROLE IF EXISTS %I', OLD.Username);
    RAISE NOTICE 'Dropped login role: %', OLD.Username;
    RETURN OLD;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_auto_drop_staff_role
AFTER DELETE ON Staff
FOR EACH ROW EXECUTE FUNCTION auto_drop_staff_role();

```

Listing 7: trg_auto_drop_staff_role — auto-drop login role on staff delete

```

CREATE OR REPLACE FUNCTION auto_update_staff_role()
RETURNS TRIGGER AS $$
DECLARE
    v_old_group VARCHAR(50);

```



```

v_new_group VARCHAR(50);
BEGIN
  IF OLD.Role = NEW.Role AND OLD.Username = NEW.Username THEN
    RETURN NEW;
  END IF;
  v_old_group := CASE OLD.Role
    WHEN 'Admin'      THEN 'admin'
    WHEN 'Ground_Staff' THEN 'ground_staff'
    WHEN 'Security'   THEN 'security_staff'
    WHEN 'Checkin'    THEN 'checkin_staff'
    ELSE NULL
  END;
  v_new_group := CASE NEW.Role
    WHEN 'Admin'      THEN 'admin'
    WHEN 'Ground_Staff' THEN 'ground_staff'
    WHEN 'Security'   THEN 'security_staff'
    WHEN 'Checkin'    THEN 'checkin_staff'
    ELSE NULL
  END;
  IF v_new_group IS NULL THEN
    RAISE EXCEPTION 'Unknown role: %.', NEW.Role;
  END IF;
  IF NOT EXISTS (SELECT 1 FROM pg_roles
    WHERE rolname = NEW.Username) THEN
    EXECUTE format('CREATE ROLE %I LOGIN PASSWORD %L',
      NEW.Username, NEW.Password_Hash);
    RAISE NOTICE 'Created login role: %', NEW.Username;
  END IF;
  IF v_old_group IS NOT NULL THEN
    EXECUTE format('REVOKE %I FROM %I',
      v_old_group, NEW.Username);
    RAISE NOTICE 'Revoked old role % from %',
      v_old_group, NEW.Username;
  END IF;
  EXECUTE format('GRANT %I TO %I', v_new_group, NEW.Username);
  RAISE NOTICE 'Granted new role % to %',
    v_new_group, NEW.Username;
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_auto_update_staff_role
AFTER UPDATE ON Staff
FOR EACH ROW EXECUTE FUNCTION auto_update_staff_role();

```

Listing 8: trg_auto_update_staff_role — auto-update login role on staff role change

7 Roles and Privileges

The system implements a **four-tier Role-Based Access Control (RBAC)** model using native PostgreSQL roles. No user accesses tables directly — all access is mediated through role grants.

7.1 Role Descriptions

admin Full access to all tables and functions. Manages flights, staff, scheduling, and system configuration.

checkin_staff Handles passenger registration, bookings, boarding pass retrieval, and luggage check-in.

security_staff Verifies boarding passes and updates luggage security status.

ground_staff Monitors gate operations, views flight schedules, and manages gate status.

7.2 Table-Level Privilege Matrix

Table	admin	checkin	security	ground
Passenger	ALL	SEL, INS	SEL	—
Staff	ALL	—	—	SEL
Airport	ALL	SEL	—	SEL
Airroutes	ALL	SEL	—	SEL
Gates	ALL	—	—	SEL, UPD
Flights	ALL	SEL	—	SEL
Scheduled_Flight	ALL	SEL	SEL	SEL, UPD
Bookings	ALL	SEL, INS, UPD	SEL	—
Boarding_Pass	ALL	SEL	SEL	—
Luggage	ALL	SEL, INS	SEL, UPD	—
Staff_Manages	ALL	—	—	SEL

7.3 Function Execution Privileges

Function	admin	checkin	security	ground
RegisterPassenger	EXEC	EXEC	—	—
GetPassengerByID	EXEC	EXEC	—	—
UpdatePassenger	EXEC	EXEC	—	—
SearchPassenger	EXEC	EXEC	—	—
StaffLogin	EXEC	EXEC	EXEC	EXEC
BookFlight	EXEC	EXEC	—	—
CancelBooking	EXEC	EXEC	—	—
GetBookingsByPassenger	EXEC	EXEC	—	—
GetBoardingPass	EXEC	EXEC	EXEC	—
AddLuggage	EXEC	EXEC	—	—
UpdateLuggageStatus	EXEC	—	EXEC	—
SearchLuggage	EXEC	EXEC	EXEC	—
ScheduleFlight	EXEC	—	—	—
UpdateFlightStatus	EXEC	—	—	EXEC
GetFlightsByDate	EXEC	—	—	EXEC
AssignGate	EXEC	—	—	EXEC
AssignStaffToFlight	EXEC	—	—	—
SearchStaff	EXEC	—	—	—
GetAllAirports	EXEC	EXEC	—	EXEC
SearchAirport	EXEC	EXEC	—	EXEC
GetGatesByAirport	EXEC	—	—	EXEC
UpdateGateStatus	EXEC	—	—	EXEC

7.4 Role Creation SQL

```
CREATE ROLE checkin_staff LOGIN PASSWORD 'check123';
CREATE ROLE security_staff LOGIN PASSWORD 'secure123';
CREATE ROLE ground_staff LOGIN PASSWORD 'ground123';

-- Table grants
GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA public TO admin;
GRANT SELECT, INSERT ON Passenger TO checkin_staff;
GRANT SELECT, INSERT ON Bookings TO checkin_staff;
GRANT SELECT ON Boarding_Pass TO checkin_staff, security_staff;
GRANT SELECT, UPDATE ON Luggage TO security_staff;
GRANT SELECT, UPDATE ON Scheduled_Flight TO ground_staff;
GRANT SELECT, UPDATE ON Gates TO ground_staff;

-- Function grants
GRANT EXECUTE ON FUNCTION GetBoardingPass(INT)
    TO checkin_staff, security_staff;
GRANT EXECUTE ON FUNCTION BookFlight(INT,INT,INT,DATE)
    TO checkin_staff;
GRANT EXECUTE ON FUNCTION AssignGate(INT,INT,DATE,INT)
```

```

    TO admin, ground_staff;
GRANT EXECUTE ON FUNCTION UpdateLuggageStatus(INT,VARCHAR)
    TO security_staff;

-- View grants
GRANT SELECT ON PassengerBookingSummary TO checkin_staff;
GRANT SELECT ON FlightGateStatus      TO ground_staff, admin;
GRANT SELECT ON LuggageSecurityView    TO security_staff;
GRANT SELECT ON StaffFlightView        TO admin, ground_staff;
GRANT SELECT ON FlightRouteView        TO admin, ground_staff,
                                         checkin_staff;
GRANT SELECT ON BookingDetailsView     TO admin, checkin_staff;
GRANT SELECT ON LuggageSummaryView     TO admin, security_staff;

```

Listing 9: Role creation and privilege grants

8 Views and Justification

The system implements **7 views** providing purpose-built interfaces for each role. Views simplify complex multi-table joins, enforce role-scoped data access, and reduce the risk of ad-hoc query errors.

8.1 View 1: PassengerBookingSummary

Access: checkin_staff, admin

Purpose: Shows each passenger alongside their booking details including flight ID, date, and status.

Justification: Eliminates repeated joins between Passenger and Bookings during check-in. Gives checkin staff a single-query view of a passenger’s travel history.

```

CREATE OR REPLACE VIEW PassengerBookingSummary AS
SELECT p.Passenger_ID, p.Name, b.Booking_ID, b.Flight_ID,
       b.Flight_Date, b.Booking_Status,
       b.Traveller_Name, b.Traveller_Email,
       b.Traveller_Phone, b.Traveller_Passport,
       b.Group_ID
FROM Passenger p
JOIN Bookings b ON p.Passenger_ID = b.Passenger_ID;

```

Listing 10: PassengerBookingSummary view

8.2 View 2: FlightGateStatus

Access: ground_staff, admin

Purpose: Shows each scheduled flight alongside its assigned gate number and gate status.

Justification: Ground staff can immediately identify gate occupancy per flight without joining Scheduled_Flight and Gates manually.

```
CREATE OR REPLACE VIEW FlightGateStatus AS
SELECT sf.Route_ID, sf.Flight_ID, sf.Flight_Date,
       sf.Gate_ID,
       g.Status AS Gate_Status,
       sf.Status AS Flight_Status
FROM Scheduled_Flight sf
JOIN Gates g ON sf.Gate_ID = g.Gate_ID;
```

Listing 11: FlightGateStatus view

8.3 View 3: LuggageSecurityView

Access: security_staff, admin

Purpose: Provides a per-passenger, per-flight aggregated summary of luggage including total bags, flagged count, cleared count, and average weight.

Justification: Security staff can instantly see which passengers have flagged luggage on a given flight without scanning individual bag records. Aggregation reduces query complexity and speeds up security workflow decisions.

```
CREATE OR REPLACE VIEW LuggageSecurityView AS
SELECT
    p.Passenger_ID,
    p.Name AS Passenger_Name,
    p.Passport_Number,
    b.Flight_ID,
    b.Flight_Date,
    COUNT(l.Luggage_ID) AS Total_Bags,
    COUNT(CASE WHEN l.Security_Status = 'Flagged'
              THEN 1 END) AS Flagged_Count,
    COUNT(CASE WHEN l.Security_Status = 'Cleared'
              THEN 1 END) AS Cleared_Count,
    ROUND(AVG(l.Weight), 2) AS Avg_Weight_KG
FROM Luggage l
JOIN Bookings b ON l.Booking_ID = b.Booking_ID
JOIN Passenger p ON b.Passenger_ID = p.Passenger_ID
GROUP BY
    p.Passenger_ID,
    p.Name,
    p.Passport_Number,
    b.Flight_ID,
    b.Flight_Date;
```

Listing 12: LuggageSecurityView view

8.4 View 4: StaffFlightView

Access: admin, ground_staff

Purpose: Shows which staff member manages which flight, with full route, aircraft, and

schedule details.

Justification: Provides a single-query staff duty roster. Avoids complex multi-table joins across Staff, Staff_Manages, Scheduled_Flight, Flights, Airroutes, and Airport.

```
CREATE OR REPLACE VIEW StaffFlightView AS
SELECT s.Staff_ID, s.Name AS Staff_Name, s.Role,
       f.Flight_No, f.Aircraft_Type,
       a1.Airport_Name AS Origin,
       a2.Airport_Name AS Destination,
       sf.Flight_Date, sf.Departure_Time, sf.Arrival_Time,
       sf.Status AS Flight_Status
FROM Staff_Manages sm
JOIN Staff          s ON sm.Staff_ID = s.Staff_ID
JOIN Scheduled_Flight sf ON sm.Route_ID = sf.Route_ID
                        AND sm.Flight_ID = sf.Flight_ID
                        AND sm.Flight_Date = sf.Flight_Date
JOIN Flights        f ON sf.Flight_ID = f.Flight_ID
JOIN Airroutes      ar ON sf.Route_ID = ar.Route_ID
JOIN Airport        a1 ON ar.Origin_Airport = a1.Airport_ID
JOIN Airport        a2 ON ar.Destination_Airport = a2.Airport_ID;
```

Listing 13: StaffFlightView view

8.5 View 5: FlightRouteView

Access: admin, ground_staff, checkin_staff

Purpose: Provides complete flight details including origin city, destination city, distance, gate number, and both flight and gate statuses.

Justification: Most operational queries need this combination of data. The view pre-joins 5 tables, making the result immediately usable by the application layer.

```
CREATE OR REPLACE VIEW FlightRouteView AS
SELECT sf.Route_ID, sf.Flight_ID,
       f.Flight_No, f.Aircraft_Type, f.Capacity,
       a1.Airport_Name AS Origin_Airport,
       a1.City          AS Origin_City,
       a2.Airport_Name AS Destination_Airport,
       a2.City          AS Destination_City,
       ar.Distance_KM,
       sf.Flight_Date, sf.Departure_Time, sf.Arrival_Time,
       g.Gate_Number,
       g.Status AS Gate_Status,
       sf.Status AS Flight_Status
FROM Scheduled_Flight sf
JOIN Flights f ON sf.Flight_ID = f.Flight_ID
JOIN Airroutes ar ON sf.Route_ID = ar.Route_ID
JOIN Airport a1 ON ar.Origin_Airport = a1.Airport_ID
JOIN Airport a2 ON ar.Destination_Airport = a2.Airport_ID
JOIN Gates g ON sf.Gate_ID = g.Gate_ID;
```

8.6 View 6: BookingDetailsView

Access: admin, checkin_staff

Purpose: Complete booking information combining passenger details, flight info, gate assignment, and boarding pass data in a single row per booking.

Justification: Replaces a 7-table join that checkin staff would need to reconstruct for every passenger query. Provides everything needed to complete check-in in one query.

```
CREATE OR REPLACE VIEW BookingDetailsView AS
SELECT b.Booking_ID, b.Booking_Date, b.Booking_Status,
       b.Traveller_Name, b.Traveller_Email,
       b.Traveller_Phone, b.Traveller_Passport,
       b.Group_ID,
       p.Passenger_ID, p.Name AS Passenger_Name,
       p.Email, p.Phone, p.Passport_Number,
       f.Flight_No, f.Aircraft_Type,
       a1.Airport_Name AS Origin_Airport,
       a1.City          AS Origin_City,
       a2.Airport_Name AS Destination_Airport,
       a2.City          AS Destination_City,
       b.Flight_Date, sf.Departure_Time, sf.Arrival_Time,
       g.Gate_Number, sf.Status AS Flight_Status,
       bp.Seat_Number, bp.Boarding_Group, bp.Boarding_Time
FROM Bookings b
JOIN Passenger   p ON b.Passenger_ID = p.Passenger_ID
JOIN Flights     f ON b.Flight_ID    = f.Flight_ID
JOIN Scheduled_Flight sf ON b.Route_ID = sf.Route_ID
                        AND b.Flight_ID = sf.Flight_ID
                        AND b.Flight_Date = sf.Flight_Date
JOIN Airroutes   ar ON b.Route_ID    = ar.Route_ID
JOIN Airport     a1 ON ar.Origin_Airport = a1.Airport_ID
JOIN Airport     a2 ON ar.Destination_Airport = a2.Airport_ID
JOIN Gates       g ON sf.Gate_ID      = g.Gate_ID
LEFT JOIN Boarding_Pass bp ON b.Booking_ID = bp.Booking_ID;
```

Listing 15: BookingDetailsView view

8.7 View 7: LuggageSummaryView

Access: admin, security_staff

Purpose: Aggregates luggage data per flight showing total count, cleared count, flagged count, average weight, and total weight.

Justification: Gives security staff a per-flight security overview without requiring complex aggregate queries. Particularly useful for identifying flights with multiple flagged items.

```

CREATE OR REPLACE VIEW LuggageSummaryView AS
SELECT f.Flight_No,
       a1.Airport_Name AS Origin,
       a2.Airport_Name AS Destination,
       sf.Flight_Date,
       COUNT(l.Luggage_ID) AS Total_Luggage,
       SUM(CASE WHEN l.Security_Status='Cleared'
                THEN 1 ELSE 0 END) AS Cleared_Count,
       SUM(CASE WHEN l.Security_Status='Flagged'
                THEN 1 ELSE 0 END) AS Flagged_Count,
       ROUND(AVG(l.Weight),2) AS Avg_Weight_KG,
       ROUND(SUM(l.Weight),2) AS Total_Weight_KG
FROM Luggage l
JOIN Bookings      b ON l.Booking_ID = b.Booking_ID
JOIN Flights       f ON b.Flight_ID  = f.Flight_ID
JOIN Scheduled_Flight sf ON b.Route_ID = sf.Route_ID
                  AND b.Flight_ID = sf.Flight_ID
                  AND b.Flight_Date = sf.Flight_Date
JOIN Airroutes      ar ON b.Route_ID  = ar.Route_ID
JOIN Airport        a1 ON ar.Origin_Airport = a1.Airport_ID
JOIN Airport        a2 ON ar.Destination_Airport = a2.Airport_ID
GROUP BY f.Flight_No, a1.Airport_Name,
         a2.Airport_Name, sf.Flight_Date;

```

Listing 16: LuggageSummaryView view

9 Indices

PostgreSQL automatically creates a B-Tree index on every PRIMARY KEY and UNIQUE constraint. The following **10 additional indices** were created explicitly to optimise frequently executed queries. Three index types are used: B-Tree, Hash, and GIN.

9.1 Index Types Used

Type	Best For	Count
B-Tree	Range queries, sorting, composite lookups	4
Hash	Exact equality (=) only	3
GIN	ILIKE / full-text pattern matching	3

9.2 B-Tree Indices

```

-- 1. Search scheduled flights by date
CREATE INDEX idx_flight_date
  ON Scheduled_Flight USING BTREE (Flight_Date);

-- 2. Search routes by origin + destination (composite)

```



```

CREATE INDEX idx_route_origin_dest
  ON Airroutes USING BTREE
  (Origin_Airport, Destination_Airport);

-- 3. Search bookings by passenger
CREATE INDEX idx_booking_passenger
  ON Bookings USING BTREE (Passenger_ID);

-- 4. Scheduled_Flight composite for route+flight lookups
CREATE INDEX idx_flight_route
  ON Scheduled_Flight USING BTREE (Route_ID, Flight_ID);

```

Listing 17: B-Tree indices

9.3 Hash Indices

```

-- 5. Filter bookings by exact status
CREATE INDEX idx_booking_status
  ON Bookings USING HASH (Booking_Status);

-- 6. Filter luggage by exact security status
CREATE INDEX idx_luggage_status
  ON Luggage USING HASH (Security_Status);

-- 7. Filter gates by exact status
CREATE INDEX idx_gate_status
  ON Gates USING HASH (Status);

```

Listing 18: Hash indices

9.4 GIN Indices

```

-- Enable pg_trgm extension required for GIN ILIKE support
CREATE EXTENSION IF NOT EXISTS pg_trgm;

-- 8. Full-text search on passenger name
CREATE INDEX idx_passenger_name_gin
  ON Passenger USING GIN (Name gin_trgm_ops);

-- 9. Full-text search on airport name
CREATE INDEX idx_airport_name_gin
  ON Airport USING GIN (Airport_Name gin_trgm_ops);

-- 10. Full-text search on staff name
CREATE INDEX idx_staff_name_gin
  ON Staff USING GIN (Name gin_trgm_ops);

```

Listing 19: GIN indices for full-text ILIKE search

9.5 Index Summary Table

Index Name	Type	Table / Column(s)	Queries Accelerated
idx_flight_date	B-Tree	Scheduled_Flight (Flight_Date)	GetFlightsByDate, SearchScheduledFlight
idx_route_origin_dest	B-Tree	Airroutes (Origin, Destination)	SearchRoutes, GetRoutesByOrigin
idx_booking_passenger	B-Tree	Bookings (Passenger_ID)	GetBookingsByPassenger, SearchBooking
idx_flight_route	B-Tree	Scheduled_Flight (Route, Flight)	ScheduleFlight, SearchStaffFlight
idx_booking_status	Hash	Bookings (Booking_Status)	SearchBooking, Query 2
idx_luggage_status	Hash	Luggage (Security_Status)	SearchLuggage, Query 3
idx_gate_status	Hash	Gates (Status)	SearchGate, ScheduleFlight
idx_passenger_name_gin	GIN	Passenger (Name)	SearchPassenger, Query 1
idx_airport_name_gin	GIN	Airport (Airport_Name)	SearchAirport, Query 5
idx_staff_name_gin	GIN	Staff (Name)	SearchStaff, Query 5

10 Five Queries Using Functions, Roles, and Indices

Each query below demonstrates the combined use of functions, role-based access control, views, and indices. Use `EXPLAIN ANALYZE` in PostgreSQL to verify index usage (look for *Index Scan* in the output plan).

10.1 Query 1: Passenger Check-In Lookup

Role: `checkin_staff` **Functions:** `SearchPassenger`, `GetBookingsByPassenger`

Indices: `idx_passenger_name_gin` (GIN), `idx_booking_passenger` (B-Tree)

View: `BookingDetailsView`

Scenario: Check-in staff searches for a passenger by name, retrieves all their bookings, and views complete booking details.

```
-- Step 1: Search passenger by name
--           (uses GIN idx_passenger_name_gin)
SELECT * FROM SearchPassenger('Priya');

-- Step 2: Get all bookings for that passenger
--           (uses B-Tree idx_booking_passenger)
SELECT * FROM GetBookingsByPassenger(2);

-- Step 3: View complete booking details
SELECT * FROM BookingDetailsView
```

```
WHERE Passenger_ID = 2;
```

Listing 20: Query 1 — Passenger check-in lookup

Index Explanation: The ILIKE in SearchPassenger triggers a GIN trigram scan on `idx_passenger_name_gin`. GetBookingsByPassenger uses `idx_booking_passenger` to retrieve all bookings for passenger 2 via a direct B-Tree index scan.

Screenshot:

```
AirportManagementsystem=> RESET ROLE;
RESET
AirportManagementsystem=# SET ROLE kavitha_r;
SET
AirportManagementsystem=> SELECT * FROM SearchPassenger('Priya');
passenger_id | name | email | phone | passport_number
-----+-----+-----+-----+-----
2 | Priya Nair | priya@gmail.com | 9876543211 | P1234568
(1 row)

AirportManagementsystem=> SELECT * FROM GetBookingsByPassenger(2);
booking_id | flight_no | origin | destination | flight_date | booking_date | booking_status
-----+-----+-----+-----+-----+-----+-----
2 | AI102 | Indira Gandhi International | Chhatrapati Shivaji Maharaj | 2024-01-15 | 2024-01-02 | Confirmed
(1 row)

AirportManagementsystem=> SELECT * FROM BookingDetailsView
AirportManagementsystem=> WHERE Passenger_ID = 2;
booking_id | booking_date | booking_status | passenger_id | passenger_name | email | phone | passport_number | flight_no | aircraft_type | origin_airport | origin_city | destination_airport | destination_city | flight_date | departure_time | arrival_time | gate_number | flight_status | seat_number | boarding_group | boarding_time
-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----
2 | 2024-01-02 | Confirmed | 2 | Priya Nair | priya@gmail.com | 9876543211 | P1234568 | AI102 | Airbus A320 | Indira Gandhi International | Delhi | Chhatrapati Shivaji Maharaj | Mumbai | 2024-01-15 | 09:00:00 | 11:30:00 | A2 | On Time | 14B | Group2 | 08:30:00
(1 row)
```

Figure 13: Query 1 — Passenger check-in lookup output in pgAdmin

10.2 Query 2: Daily Flight Schedule + Booking Count

Role: `ground_staff`, `admin` **Functions:** `GetFlightsByDate`, `SearchScheduledFlight`

Indices: `idx_flight_date` (B-Tree), `idx_booking_status` (Hash)

Scenario: Ground staff checks all flights on a date, filters delayed ones, and counts confirmed bookings per flight.

```
-- Step 1: Get all flights on a date
--           (uses B-Tree idx_flight_date)
SELECT * FROM GetFlightsByDate('2024-01-15');

-- Step 2: Filter only delayed flights
SELECT * FROM SearchScheduledFlight(
    p_date => '2024-01-15',
    p_status => 'Delayed',
    p_route => NULL
);

-- Step 3: Count confirmed bookings per flight
--           (uses Hash idx_booking_status, B-Tree idx_flight_date)
```

```

SELECT f.Flight_No,
       COUNT(b.Booking_ID) AS Confirmed_Bookings
FROM Bookings b
JOIN Flights f ON b.Flight_ID = f.Flight_ID
WHERE b.Booking_Status = 'Confirmed'
      AND b.Flight_Date = '2024-01-15'
GROUP BY f.Flight_No
ORDER BY Confirmed_Bookings DESC;

```

Listing 21: Query 2 — Daily flight schedule

Index Explanation: `idx_flight_date` handles the date equality filter in `GetFlightsByDate`. The `Booking_Status = 'Confirmed'` exact match in Step 3 hits the Hash index `idx_booking_status` directly.

Screenshots:

```
AirportManagementsystem=# SET ROLE prakash_n;
```

```
SET
```

```
AirportManagementsystem=> SELECT * FROM GetFlightsByDate('2024-01-15');
```

route_id	flight_id	flight_no	origin	destination	flight_date	departure_time	arrival_time	gate_number	status
1	1	AI101	Indira Gandhi International	Chhatrapati Shivaji Maharaj	2024-01-15	06:00:00	08:00:00	A1	On_Time
8	9	AI109	Kempegowda International	Chennai International	2024-01-15	06:30:00	08:15:00	C1	On_Time
2	3	AI103	Indira Gandhi International	Kempegowda International	2024-01-15	07:00:00	09:45:00	C1	Delayed
15	16	UK401	Sardar Vallabhbhai Patel	Indira Gandhi International	2024-01-15	07:30:00	09:00:00	H1	On_Time
5	6	AI106	Chhatrapati Shivaji Maharaj	Kempegowda International	2024-01-15	08:00:00	09:45:00	B1	On_Time
1	2	AI102	Indira Gandhi International	Chhatrapati Shivaji Maharaj	2024-01-15	09:00:00	11:30:00	A2	On_Time
9	10	AI110	Kempegowda International	Rajiv Gandhi International	2024-01-15	09:30:00	11:00:00	D1	Delayed
3	4	AI104	Indira Gandhi International	Chennai International	2024-01-15	10:00:00	14:00:00	D1	On_Time
16	17	UK402	Sardar Vallabhbhai Patel	Chhatrapati Shivaji Maharaj	2024-01-15	10:30:00	12:00:00	I1	On_Time
4	5	AI105	Indira Gandhi International	Rajiv Gandhi International	2024-01-15	11:00:00	13:30:00	E1	On_Time
17	18	G8501	Pune Airport	Indira Gandhi International	2024-01-15	12:00:00	14:00:00	A1	On_Time
6	7	AI107	Chhatrapati Shivaji Maharaj	Chennai International	2024-01-15	13:00:00	15:20:00	F1	Cancelled
7	8	AI108	Chhatrapati Shivaji Maharaj	Cochin International	2024-01-15	14:00:00	17:00:00	G1	On_Time
18	19	G8502	Pune Airport	Kempegowda International	2024-01-15	15:00:00	16:45:00	C1	On_Time
10	11	6E201	Chennai International	Rajiv Gandhi International	2024-01-15	16:00:00	17:10:00	E1	On_Time
11	12	6E202	Chennai International	Cochin International	2024-01-15	17:00:00	19:30:00	F1	On_Time

```
AirportManagementsystem=> SELECT * FROM SearchScheduledFlight(
```

```
AirportManagementsystem(> p_date => '2024-01-15',
```

```
AirportManagementsystem(> p_status => 'Delayed',
```

```
AirportManagementsystem(> p_route => NULL
```

```
AirportManagementsystem(> );
```

route_id	flight_id	flight_no	origin	destination	flight_date	departure_time	arrival_time	status
2	3	AI103	Indira Gandhi International	Kempegowda International	2024-01-15	07:00:00	09:45:00	Delayed
9	10	AI110	Kempegowda International	Rajiv Gandhi International	2024-01-15	09:30:00	11:00:00	Delayed
14	15	SG302	Netaji Subhas Chandra Bose	Indira Gandhi International	2024-01-15	20:00:00	21:35:00	Delayed

(3 rows)

```

AirportManagementsystem=> RESET ROLE;
RESET
AirportManagementsystem=# SET ROLE kavitha_r;
SET
AirportManagementsystem=> SELECT f.Flight_No,
AirportManagementsystem-> COUNT(b.Booking_ID) AS Confirmed_Bookings
AirportManagementsystem-> FROM Bookings b
AirportManagementsystem-> JOIN Flights f ON b.Flight_ID = f.Flight_ID
AirportManagementsystem-> WHERE b.Booking_Status = 'Confirmed'
AirportManagementsystem-> AND b.Flight_Date = '2024-01-15'
AirportManagementsystem-> GROUP BY f.Flight_No
AirportManagementsystem-> ORDER BY Confirmed_Bookings DESC;
flight_no | confirmed_bookings
-----+-----
AI102    |                4
AI101    |                4
6E203    |                1
AI103    |                1
AI105    |                1
AI106    |                1
AI107    |                1
AI108    |                1
AI109    |                1
AI110    |                1
G8501    |                1
G8502    |                1
IX601    |                1
SG302    |                1
UK401    |                1
6E201    |                1
UK402    |                1
6E202    |                1
(18 rows)

```

10.3 Query 3: Security Luggage Inspection

Role: security_staff **Functions:** SearchLuggage, GetLuggageByBooking, UpdateLuggageStatus

Index: idx_luggage_status (Hash) **View:** LuggageSecurityView

Scenario: Security staff finds all flagged luggage, inspects a specific booking's bags, and clears them after inspection.

```

-- Step 1: Find all flagged luggage
--           (uses Hash idx_luggage_status)
SELECT * FROM SearchLuggage(
    p_booking_id => NULL,
    p_status     => 'Flagged'
);

-- Step 2: View flagged summary per flight
SELECT * FROM LuggageSecurityView
WHERE Flagged_Count > 0;

-- Step 3: Get details for a specific booking
SELECT * FROM GetLuggageByBooking(3);

-- Step 4: Clear flagged luggage after inspection
SELECT UpdateLuggageStatus(3, 'Cleared');

```

Listing 22: Query 3 — Luggage security inspection

Index Explanation: The `Security_Status = 'Flagged'` equality filter in `SearchLuggage` hits the Hash index `idx_luggage_status`, returning flagged records in $O(1)$ average time without a sequential scan.

Screenshots:

```
AirportManagementsystem=> RESET ROLE;
RESET
AirportManagementsystem=# SET ROLE preethi_n;
SET
AirportManagementsystem=> SELECT * FROM SearchLuggage(
AirportManagementsystem(> p_booking_id => NULL,
AirportManagementsystem(> p_status => 'Flagged'
AirportManagementsystem(> );
```

luggage_id	booking_id	weight	security_status	checkin_time
16	18	14.00	Flagged	2024-01-15 11:00:00
19	26	15.00	flagged	2026-04-19 16:24:51.766
1	1	23.50	Flagged	2024-01-15 04:30:00
2	2	18.00	Flagged	2024-01-15 07:45:00
4	5	15.50	Flagged	2024-01-15 09:00:00
5	6	22.00	Flagged	2024-01-15 06:30:00
6	7	10.00	Flagged	2024-01-15 11:00:00

(7 rows)

Figure 14: Query 3 — Step 1: `SearchLuggage` showing all Flagged luggage

```
AirportManagementsystem=# SELECT * FROM LuggageSecurityView
AirportManagementsystem=# WHERE Flagged_Count>0;
passenger_id | passenger_name | passport_number | flight_id | flight_date | total_bags | flagged_count | cleared_count
| avg_weight_kg
```

passenger_id	passenger_name	passport_number	flight_id	flight_date	total_bags	flagged_count	cleared_count
6	Divya Menon	P1234572	6	2024-01-15	1	1	0
2	Priya Nair	P1234568	2	2024-01-15	1	1	0
7	Arjun Singh	P1234573	7	2024-01-15	1	1	0
5	Kiran Kumar	P1234571	5	2024-01-15	1	1	0
1	Rahul Sharma	P1234567	1	2024-01-15	2	1	0
18	Deepa Krishnan	P1234584	18	2024-01-15	1	1	0

(6 rows)

Activate Window:

Figure 15: Query 3 — Step 2: `GetLuggageByBooking` output for booking 3


```

AirportManagementsystem=# SELECT * FROM GetLuggageByBooking(3);
 luggage_id | booking_id | weight | security_status | checkin_time
-----+-----+-----+-----+-----
          3 |          3 |  30.00 | Cleared         | 2024-01-15 05:30:00
(1 row)

AirportManagementsystem=# SELECT UpdateLuggageStatus(3, 'Cleared');
 updateluggagestatus
-----
(1 row)

AirportManagementsystem=#

```

Figure 16: Query 3 — Step 3 4: UpdateLuggageStatus clearing flagged luggage

10.4 Query 4: Gate Management and Flight Scheduling

Role: admin, ground_staff **Functions:** SearchRoutes, SearchGate, ScheduleFlight

Indices: idx_route_origin_dest (B-Tree), idx_gate_status (Hash), idx_flight_date (B-Tree)

Views: FlightRouteView, FlightGateStatus

Scenario: Admin searches for a route, finds an available gate, and schedules a new flight.

```

-- Step 1: Search routes Delhi -> Mumbai
--      (uses B-Tree idx_route_origin_dest)
SELECT * FROM SearchRoutes(p_origin => 1, p_destination => 2);

-- Step 2: View flights on target date
--      (uses B-Tree idx_flight_date)
SELECT * FROM FlightRouteView
WHERE Flight_Date = '2024-01-15'
ORDER BY Departure_Time;

-- Step 3: Schedule new flight
CALL ScheduleFlight(
    p_route_id => 1,
    p_flight_id => 1,
    p_flight_date => '2024-01-20',
    p_departure => '08:00',
    p_arrival   => '10:00',
    p_gate_id   => 1,
    p_status    => 'On_Time'
);

```

Listing 23: Query 4 — Gate management and scheduling

Index Explanation: The two-column WHERE on Origin and Destination uses the composite `idx_route_origin_dest`. The exact status filter uses Hash `idx_gate_status`. Viewing existing schedules on a date uses `idx_flight_date`.

Screenshots:

```
AirportManagementsystem=# SET ROLE mohan_d;
SET
AirportManagementsystem=> SELECT * FROM SearchRoutes(p_origin => 1, p_destination => 2);
 route_id | origin_airport_name | destination_airport_name | distance_km
-----+-----+-----+-----
          1 | Indira Gandhi International | Chhatrapati Shivaji Maharaj |          1150
(1 row)
```

Figure 17: Query 4 — Step 1: SearchRoutes output for Delhi to Mumbai

```
AirportManagementsystem=> SELECT * FROM FlightRouteView
AirportManagementsystem-> WHERE Flight_Date = '2024-01-15'
AirportManagementsystem-> ORDER BY Departure_Time;
 route_id | flight_id | flight_no | aircraft_type | capacity | origin_airport | origin_city | destination_airport | destination_city | distance_km | flight_date | departure_time | arrival_time | gate_number | gate_status
-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----
          1 |          1 | AI101 | Boeing 737 |      180 | Indira Gandhi International | Delhi | Chhatrapati Shivaji Maharaj | Mumbai |      1150 | 2024-01-15 | 06:00:00 | 08:00:00 | A1 | Availab
le | On_Time
          8 |          9 | AI109 | Boeing 737 |      180 | Kempegowda International | Bangalore | Chennai International | Chennai |     1060 | 2024-01-15 | 06:30:00 | 08:15:00 | C1 | Availab
le | On_Time
          2 |          3 | AI103 | Boeing 777 |      300 | Indira Gandhi International | Delhi | Kempegowda International | Bangalore |     1740 | 2024-01-15 | 07:00:00 | 09:45:00 | C1 | Availab
le | Delayed
        15 |         16 | UK401 | ATR 72 |       70 | Sardar Vallabhbhai Patel | Ahmedabad | Indira Gandhi International | Delhi |       950 | 2024-01-15 | 07:30:00 | 09:00:00 | H1 | Availab
le | On_Time
          5 |          6 | AI106 | Airbus A320 |      160 | Chhatrapati Shivaji Maharaj | Mumbai | Kempegowda International | Bangalore |       980 | 2024-01-15 | 08:00:00 | 09:45:00 | B1 | Availab
le | On_Time
          1 |          2 | AI102 | Airbus A320 |      160 | Indira Gandhi International | Delhi | Chhatrapati Shivaji Maharaj | Mumbai |     1150 | 2024-01-15 | 09:00:00 | 11:30:00 | A2 | Occupie
d | On_Time
          9 |         10 | AI110 | Airbus A320 |      160 | Kempegowda International | Bangalore | Rajiv Gandhi International | Hyderabad |       570 | 2024-01-15 | 09:30:00 | 11:00:00 | D1 | Availab
le | Delayed
          3 |          4 | AI104 | Airbus A380 |      400 | Indira Gandhi International | Delhi | Chennai International | Chennai |     2180 | 2024-01-15 | 10:00:00 | 14:00:00 | D1 | Availab
le | On_Time
        16 |         17 | UK402 | ATR 72 |       70 | Sardar Vallabhbhai Patel | Ahmedabad | Chhatrapati Shivaji Maharaj | Mumbai |       530 | 2024-01-15 | 10:30:00 | 12:00:00 | I1 | Availab
le | On_Time
          4 |          5 | AI105 | Boeing 737 |      180 | Indira Gandhi International | Delhi | Rajiv Gandhi International | Hyderabad |     1250 | 2024-01-15 | 11:00:00 | 13:30:00 | E1 | Availab
le | On_Time
```

Figure 18: Query 4 — Step 2: SearchGate showing Available gates


```

AirportManagementsystem=> RESET ROLE;
RESET
AirportManagementsystem=# SET ROLE admin;
SET
AirportManagementsystem=> CALL ScheduleFlight(
AirportManagementsystem(> p_route_id => 1,
AirportManagementsystem(> p_flight_id => 1,
AirportManagementsystem(> p_flight_date => '2024-01-20',
AirportManagementsystem(> p_departure => '08:00',
AirportManagementsystem(> p_arrival => '10:00',
AirportManagementsystem(> p_gate_id => 1,
AirportManagementsystem(> p_status => 'On_Time'
AirportManagementsystem(> );
NOTICE: Flight scheduled successfully.
CALL

```

Figure 19: Query 4 — Step 4: ScheduleFlight confirmation in pgAdmin

10.5 Query 5: Staff Assignment and Flight Status Update

Role: admin **Functions:** SearchStaff, SearchAirport, AssignStaffToFlight, SearchStaffFlight, UpdateFlightStatus

Indices: idx_staff_name_gin (GIN), idx_airport_name_gin (GIN), idx_flight_date (B-Tree), idx_flight_route (B-Tree)

View: StaffFlightView

Scenario: Admin searches for a staff member by name, assigns them to a flight, verifies the assignment, and updates the flight status.

```

-- Step 1: Search staff by name
--      (uses GIN idx_staff_name_gin)
SELECT * FROM SearchStaff('Prakash');

-- Step 2: Search airport by name
--      (uses GIN idx_airport_name_gin)
SELECT * FROM SearchAirport('Indira');

-- Step 3: Assign staff to flight
CALL AssignStaffToFlight(
    p_staff_id => 3,
    p_route_id => 1,
    p_flight_id => 1,
    p_flight_date => '2024-01-15'
);

-- Step 4: Verify assignment
--      (uses B-Tree idx_flight_date, idx_flight_route)
SELECT * FROM SearchStaffFlight(
    p_staff_id => 3,
    p_flight_date => '2024-01-15'

```

```

);

-- Step 5: View full staff-flight summary
SELECT * FROM StaffFlightView
WHERE Flight_Date = '2024-01-15'
ORDER BY Staff_Name;

-- Step 6: Update flight status
CALL UpdateFlightStatus(
    p_route_id => 1,
    p_flight_id => 1,
    p_flight_date => '2024-01-15',
    p_status => 'Departed'
);

-- Step 7: Review status change audit log
SELECT * FROM Flight_Status_Log
WHERE Flight_ID = 1
ORDER BY Changed_At DESC;

```

Listing 24: Query 5 — Staff assignment and status update

Index Explanation: ILIKE searches in SearchStaff and SearchAirport use GIN indices `idx_staff_name_gin` and `idx_airport_name_gin`. SearchStaffFlight uses `idx_flight_date` for date filtering and `idx_flight_route` for the route+flight composite lookup.

Screenshots:

```

AirportManagementsystem=> SELECT * FROM SearchStaff('Prakash');
staff_id | name       | username | role       | address
-----+-----+-----+-----+-----
3 | Prakash Nair | prakash_n | Ground_Staff | 78 Nehru St, Kochi, Kerala
(1 row)

AirportManagementsystem=> SELECT * FROM SearchAirport('Indira');
airport_id | airport_name | city | state | country
-----+-----+-----+-----+-----
1 | Indira Gandhi International | Delhi | Delhi | India
(1 row)

AirportManagementsystem=> CALL AssignStaffToFlight(
AirportManagementsystem(> p_staff_id => 3,
AirportManagementsystem(> p_route_id => 1,
AirportManagementsystem(> p_flight_id => 1,
AirportManagementsystem(> p_flight_date => '2024-01-15'
AirportManagementsystem(> );
NOTICE: Staff 3 assigned to flight successfully.
CALL

```

Figure 20: Query 5 — Step 1: SearchStaff output for name 'Prakash'

```
AirportManagementsystem=> SELECT * FROM SearchStaffFlight(
AirportManagementsystem(> p_staff_id => 3,
AirportManagementsystem(> p_flight_date => '2024-01-15'
AirportManagementsystem(> );
```

	staff_id	staff_name	role	flight_no	origin	destination	flight_date
01-15	3	Prakash Nair	Ground_Staff	AI101	Indira Gandhi International	Chhatrapati Shivaji Maharaj	2024-01-15
	3	Prakash Nair	Ground_Staff	AI103	Indira Gandhi International	Kempegowda International	2024-01-15
	3	Prakash Nair	Ground_Staff	6E203	Rajiv Gandhi International	Cochin International	2024-01-15

(3 rows)

Figure 21: Query 5 — Step 3: AssignStaffToFlight confirmation

```
AirportManagementsystem=> SELECT * FROM StaffFlightView
AirportManagementsystem-> WHERE Flight_Date = '2024-01-15'
AirportManagementsystem-> ORDER BY Staff_Name;
```

	staff_id	staff_name	role	flight_no	aircraft_type	origin	destination
onal	4	Anita Rao	Security	AI104	Airbus A380	Indira Gandhi International	Chennai International
	4	Anita Rao	Security	SG301	Boeing 737-800	Cochin International	Netaji Subhas Chandra Bose
ational	9	Arun Menon	Ground_Staff	G8502	Airbus A320	Pune Airport	Kempegowda International
	9	Arun Menon	Ground_Staff	AI109	Boeing 737	Kempegowda International	Chennai International
onal	10	Kavitha Reddy	Admin	IX601	Airbus A320	Trivandrum International	Chennai International
	10	Kavitha Reddy	Admin	AI110	Airbus A320	Kempegowda International	Rajiv Gandhi International
rnational	7	Mohan Das	Ground_Staff	UK402	ATR 72	Sardar Vallabhbhai Patel	Chhatrapati Shivaji Maharaj
	7	Mohan Das	Ground_Staff	AI107	Boeing 737	Chhatrapati Shivaji Maharaj	Chennai International
onal	3	Prakash Nair	Ground_Staff	AI101	Boeing 737	Indira Gandhi International	Chhatrapati Shivaji Maharaj
	3	Prakash Nair	Ground_Staff	AI103	Boeing 777	Indira Gandhi International	Kempegowda International
ational	3	Prakash Nair	Ground_Staff	6E203	Boeing 737	Rajiv Gandhi International	Cochin International
	8	Preethi Nair	Security	G8501	Boeing 737	Pune Airport	Indira Gandhi International
ernational	8	Preethi Nair	Security	AI108	Airbus A321	Chhatrapati Shivaji Maharaj	Cochin International
	1	Ramesh Kumar	Admin	6E201	Airbus A320	Chennai International	Rajiv Gandhi International
rnational	1	Ramesh Kumar	Admin	AI101	Boeing 737	Indira Gandhi International	Chhatrapati Shivaji Maharaj
	6	Rekha Iyer	Admin	AI106	Airbus A320	Chhatrapati Shivaji Maharaj	Kempegowda International

Figure 22: Query 5 — Step 4 & 5: SearchStaffFlight and StaffFlightView output

```
AirportManagementsystem=> CALL UpdateFlightStatus(
AirportManagementsystem(> p_route_id => 1,
AirportManagementsystem(> p_flight_id => 1,
AirportManagementsystem(> p_flight_date => '2024-01-15',
AirportManagementsystem(> p_status => 'Departed'
AirportManagementsystem(> );
NOTICE: Flight status changed from On_Time to Departed for Flight 1 on Route 1 dated 2024-01-15.
NOTICE: Flight status updated to Departed.
CALL
AirportManagementsystem=> SELECT * FROM Flight_Status_Log
AirportManagementsystem-> WHERE Flight_ID = 1
AirportManagementsystem-> ORDER BY Changed_At DESC;
```

	log_id	route_id	flight_id	flight_date	old_status	new_status	changed_at
(1 row)	1	1	1	2024-01-15	On_Time	Departed	2026-05-04 20:19:32.394443

Figure 23: Query 5 — Step 6 & 7: UpdateFlightStatus and Flight_Status_Log audit

11 Primary Tasks on the Data

- Register passengers and manage booking records
- Schedule flights and assign gates
- Automatically generate boarding passes upon booking confirmation
- Track and validate luggage details and security clearance
- Enforce data integrity using constraints, foreign keys, and triggers
- Support role-based access control for all staff roles
- Maintain air route data with origin and destination airport references
- Track which flights follow which routes via the `Scheduled.Flight` mapping table

Index

Aggregations, [6](#)
Airport Management System, [3](#)
Airports, [4](#)
Airroutes, [4](#)

Boarding Pass, [3](#), [4](#)
Bookings, [4](#)

Constraints, [13](#)

Data, [3](#)

ER Diagram, [5](#)

Flight scheduling, [3](#)
Flights, [4](#)
Functions, [15](#)

Gates, [3](#), [4](#)

Indices, [32](#)

Luggage, [3](#), [4](#)

Passenger, [3](#), [4](#)
Primary Tasks, [44](#)

Queries, [34](#)

Relational Tables, [4](#)
Relationships, [5](#)

Scheduled Flight, [4](#)
Staff, [4](#)
Staff Manages, [4](#)

Table Creation, [6](#)
Triggers, [3](#)

User Roles, [26](#)

Views, [28](#)